# INSTITUTO POLITÉCNICO NACIONAL

## UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

**UNIDAD DE APRENDIZAJE**

Ingeniería de Pruebas

**EXAMEN**

Caja Negra

**SECUENCIA Y PERIODO**

6NM61 2026-2

**BOLETA**

2021601769

**ALUMNO**

Gonzalez Calzada Maximiliano

**PROFESORA**

González Arroyo Lilia

**FECHA**

31 de Marzo de 2026

# Caja Negra Individual

## 1. Ejercicio 1: Aplicación Financiera

Se definen las particiones de equivalencia para las variables Principal, Tasa de Interés y Estimación de Inflación. Este ejercicio aplica pruebas de **caja negra**: no se analiza la lógica interna del sistema, únicamente se verifica que ante determinadas entradas el sistema produzca las salidas correctas. El reto es que el espacio de posibles combinaciones de tres valores numéricos es infinito, por lo que se utilizan técnicas sistemáticas para seleccionar un subconjunto finito pero representativo de casos de prueba.

### 1.1. Criterio de obtención de resultados

Los resultados esperados se derivan con **partición equivalente**:

- Definir dominio válido por variable.
- Separar clases válidas e inválidas.
- Elegir un valor representativo por clase.
- Asignar el resultado esperado según la regla de validación.

La idea central es que si un valor dentro de una clase produce el resultado correcto, todos los demás valores de esa misma clase producirán el mismo resultado. Así, basta probar **un solo representante por clase** en lugar de probar todos los valores posibles. Para cada variable se identifican tres clases: válida (dentro del rango permitido), inválida por debajo del mínimo e inválida por encima del máximo.

Además, se incluyen casos de **valores límite (BVA)** para validar fronteras: `min-1`, `min`, `min+1`, `max-1`, `max`, `max+1`.

La experiencia en pruebas indica que los errores tienden a concentrarse en los bordes de los rangos, no en los valores centrales. Por ello, el análisis de valor frontera complementa la partición equivalente probando específicamente los valores exactos de la frontera, el inmediato interior y el inmediato exterior de cada límite.

## 1.2. Origen de los resultados esperados

- `100 ⩽ principal ⩽ 100_000_000 → Válido`
- `principal < 100 → Error: Valor mínimo 100`
- `principal > 100_000_000 → Error: Valor máximo 100M`
- `1 ⩽ inflacion ⩽ 20 → Válido`
- `inflacion < 1 → Error: Valor mínimo 1%`
- `inflacion > 20 → Error: Valor máximo 20%`
- `0 ⩽ interes ⩽ 25 → Válido`
- `interes < 0 → Error: Valor mínimo 0%`
- `interes > 25 → Error: Valor máximo 25%`

## 1.3. Código hipotético que produce esos resultados

```typescript
type Resultado = "Válido" | string

function validarAplicacionFinanciera(
  principal: number,
  inflacion: number,
  interes: number,
): Resultado {
  if (principal < 100) return "Error: Valor mínimo 100"
  if (principal > 100_000_000) return "Error: Valor máximo 100M"

  if (inflacion < 1) return "Error: Valor mínimo 1%"
  if (inflacion > 20) return "Error: Valor máximo 20%"

  if (interes < 0) return "Error: Valor mínimo 0%"
  if (interes > 25) return "Error: Valor máximo 25%"

  return "Válido"
}
```

La siguiente tabla muestra los 9 casos de prueba resultantes: tres por cada variable, uno por cada clase de equivalencia. Cada fila es independiente, es decir, al probar el valor de una variable se asume que las demás están en su rango válido nominal.

| Variable | Partición | Valor de Prueba | Resultado Esperado |
|---|---|---|---|
| Principal | Válida: (100, 100M) | 1,000,000 | Válido |
| Principal | Inválida: < 100 | 50 | Error: Valor mínimo 100 |
| Principal | Inválida: > 100M | 150,000,000 | Error: Valor máximo 100M |
| Inflación | Válida: (1, 20) | 5 | Válido |
| Inflación | Inválida: < 1 | 0.5 | Error: Valor mínimo 1% |
| Inflación | Inválida: > 20 | 25 | Error: Valor máximo 20% |
| Interés | Válida: (0, 25) | 10 | Válido |
| Interés | Inválida: < 0 | –5 | Error: Valor mínimo 0% |
| Interés | Inválida: > 25 | 30 | Error: Valor máximo 25% |

# 2. Ejercicio 2: Función Matemática F(x)

Se definen las particiones basadas en los subdominios de la función real.

## 2.1. Valores límite (BVA)

Para complementar la partición equivalente, se aplica el análisis de valor frontera sobre las mismas tres variables. Se prueba cada límite con tres valores: justo fuera (`min-1`, `max+1`), en la frontera exacta (`min`, `max`) y justo dentro (`min+1`, `max-1`). Para aislar el efecto de cada variable, las demás se mantienen en un valor nominal válido: `principal=1,000,000`, `inflacion=5`, `interes=10`.

| Variable | Valor límite | Resultado Esperado |
|---|---|---|
| Principal | 99 | Error: Valor mínimo 100 |
| Principal | 100 | Válido |
| Principal | 101 | Válido |
| Principal | 99,999,999 | Válido |

| | | |
|---|---|---|
| Principal | 100,000,000 | Válido |
| Principal | 100,000,001 | Error: Valor máximo 100M |
| Inflación | 0.99 | Error: Valor mínimo 1% |
| Inflación | 1 | Válido |
| Inflación | 1.01 | Válido |
| Inflación | 19.99 | Válido |
| Inflación | 20 | Válido |
| Inflación | 20.01 | Error: Valor máximo 20% |
| Interés | –0.01 | Error: Valor mínimo 0% |
| Interés | 0 | Válido |
| Interés | 0.01 | Válido |
| Interés | 24.99 | Válido |
| Interés | 25 | Válido |
| Interés | 25.01 | Error: Valor máximo 25% |

### 2.2. Origen de los resultados esperados

- `x < -2 → Procesar en Intervalo 1`
- `-2 ⩽ x ⩽ 4 → Procesar en Intervalo 2`
- `x > 4 → Procesar en Intervalo 3`

### 2.3. Código hipotético que produce esos resultados

```typescript
function clasificarIntervaloFx(x: number): string {
  if (x < -2) return "Procesar en Intervalo 1"
  if (x ⩽ 4) return "Procesar en Intervalo 2"
  return "Procesar en Intervalo 3"
}
```

| Clase de Equivalencia | Valor de Prueba (x) | Resultado Esperado |
|---|---|---|
| Clase 1: $(-\infty, -2)$ | –10 | Procesar en Intervalo 1 |
| Clase 2: $[-2, 4]$ | 0 | Procesar en Intervalo 2 |
| Clase 3: $(4, +\infty)$ | 10 | Procesar en Intervalo 3 |

## 3. Ejercicio 3: Validación de Fechas

Se analizan las particiones para los campos de Día, Mes y Año, considerando años bisiestos y meses de diferente duración.

### 3.1. Valores límite (BVA)

| Frontera | Valor (x) | Resultado Esperado |
|---|---|---|
| –2 | –2.01 | Procesar en Intervalo 1 |
| –2 | –2 | Procesar en Intervalo 2 |
| –2 | –1.99 | Procesar en Intervalo 2 |
| 4 | 3.99 | Procesar en Intervalo 2 |
| 4 | 4 | Procesar en Intervalo 2 |
| 4 | 4.01 | Procesar en Intervalo 3 |

### 3.2. Origen de los resultados esperados

Reglas aplicadas:

- `1900 ⩽ anio ⩽ 2199`
- `1 ⩽ mes ⩽ 12`
- `dia ⩾ 1`
- `dia ⩽ maxDia(mes, anio)`
- En febrero: `maxDia = 29` si el año es bisiesto, en otro caso `28`.

### 3.3. Código hipotético que produce esos resultados

```typescript
function esBisiesto(anio: number): boolean {
  return anio % 400 === 0 || (anio % 4 === 0 && anio % 100 !== 0)
}

function validarFecha(dia: number, mes: number, anio: number): string {
  if (anio < 1900) return "Error: Año fuera de rango (min 1900)"
  if (anio > 2199) return "Error: Año fuera de rango (max 2199)"
  if (mes < 1 || mes > 12) return "Error: Mes fuera de rango"
  if (dia < 1) return "Error: Día fuera de rango"

  const diasPorMes = [31, esBisiesto(anio) ? 29 : 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
  const maxDia = diasPorMes[mes - 1]

  if (dia > maxDia) {
    if (mes === 2 && dia === 29) return `Error: ${anio} no es bisiesto`
    if (mes === 6 && dia === 31) return "Error: Junio máximo 30 días"
    return "Error: Día fuera de rango"
  }

```

```
  if (mes === 2) return "Fecha Válida (Febrero)"
  if (maxDia === 31) return "Fecha Válida (31 días)"
  return "Fecha Válida (30 días)"
}
```

| Mes | Día | Año | Resultado Esperado |
|---|---|---|---|
| 1 | 15 | 2000 | Fecha Válida (31 días) |
| 4 | 15 | 2000 | Fecha Válida (30 días) |
| 2 | 15 | 2000 | Fecha Válida (Febrero) |
| 0 | 1 | 2000 | Error: Mes fuera de rango |
| 13 | 1 | 2000 | Error: Mes fuera de rango |
| 6 | 0 | 2000 | Error: Día fuera de rango |
| 6 | 31 | 2000 | Error: Junio máximo 30 días |
| 2 | 29 | 2001 | Error: 2001 no es bisiesto |
| 1 | 1 | 1800 | Error: Año fuera de rango (min 1900) |
| 1 | 1 | 2300 | Error: Año fuera de rango (max 2199) |

### 3.4. Valores límite (BVA)

| Campo | Entrada (día, mes, año) | Resultado Esperado | Límite |
|---|---|---|---|
| Mes | (1, 0, 2000) | Error: Mes fuera de rango | min-1 |
| Mes | (1, 1, 2000) | Fecha Válida (31 días) | min |
| Mes | (1, 12, 2000) | Fecha Válida (31 días) | max |
| Mes | (1, 13, 2000) | Error: Mes fuera de rango | max+1 |
| Día (mes 31) | (0, 1, 2000) | Error: Día fuera de rango | min-1 |
| Día (mes 31) | (1, 1, 2000) | Fecha Válida (31 días) | min |
| Día (junio) | (30, 6, 2000) | Fecha Válida (30 días) | max |

| | | | |
|---|---|---|---|
| Día (junio) | (31, 6, 2000) | Error: Junio máximo 30 días | max+1 |
| Día (febrero bisiesto) | (29, 2, 2000) | Fecha Válida (Febrero) | max |
| Día (febrero no bisiesto) | (29, 2, 2001) | Error: 2001 no es bisiesto | max+1 lógico |
| Año | (1, 1, 1899) | Error: Año fuera de rango (min 1900) | min-1 |
| Año | (1, 1, 1900) | Fecha Válida (31 días) | min |
| Año | (1, 1, 2199) | Fecha Válida (31 días) | max |
| Año | (1, 1, 2200) | Error: Año fuera de rango (max 2199) | max+1 |

## 4. Matriz de trazabilidad (Regla → Caso → Resultado)

| **Ejercicio** | **Regla** | **Caso representativo** | **Resultado Esperado** |
|---|---|---|---|
| E1 | Principal < 100 | principal=50 | Error: Valor mínimo 100 |
| E1 | Principal > 100M | principal=150,000,000 | Error: Valor máximo 100M |
| E1 | Inflación < 1 | inflación=0.5 | Error: Valor mínimo 1% |
| E1 | Inflación > 20 | inflación=25 | Error: Valor máximo 20% |
| E1 | Interés < 0 | interés=-5 | Error: Valor mínimo 0% |
| E1 | Interés > 25 | interés=30 | Error: Valor máximo 25% |

| | | | |
|---|---|---|---|
| E2 | x < –2 | x=-10 | Procesar en Intervalo 1 |
| E2 | –2 <= x <= 4 | x=0 | Procesar en Intervalo 2 |
| E2 | x > 4 | x=10 | Procesar en Intervalo 3 |
| E3 | Mes fuera de rango | (1, 13, 2000) | Error: Mes fuera de rango |
| E3 | Día excede máximo mensual | (31, 6, 2000) | Error: Junio máximo 30 días |
| E3 | 29/feb en no bisiesto | (29, 2, 2001) | Error: 2001 no es bisiesto |
| E3 | Año fuera de rango | (1, 1, 2300) | Error: Año fuera de rango (max 2199) |